



COURSE CODE:- CSE2006

COURSE NAME: Programming in Java

Fall Semester 2026-27

PROJECT REPORT FILE

**Subject - Metro Management System with
Smart Card Integration**

SUBMITTED BY:-

Saanvi Shukla

25BAI10111

INDEX

S. No.	Contents	Page No.
1	Cover Page	
2	Introduction	2
3	Problem Statement	3
4	Functional Requirements	4
5	Non-functional Requirements	5
6	System Architecture	6
7	Design Diagrams	7-10
7.1	Use Case Diagram	
7.2	Workflow Diagram	
7.3	Sequence Diagram	
7.4	Class/Component Diagram	
7.5	ER Diagram	
8	Design Decisions & Rationale	11
9	Implementation Details	
10	Screenshots / Results	14-16
11	Testing Approach	17
12	Challenges Faced	18
13	Learnings & Key Takeaways	19
14	Future Enhancements	21
15	References	21

2. Introduction

The Metro Management System with Smart Card Integration is a command-line based Java application that models a simplified digital metro travel workflow. The system combines Core Java, object-oriented programming, JDBC and MySQL to manage passengers, smart cards, stations, journeys and fare transactions.

A passenger can register or use the supplied demo account, obtain an active smart card, view its balance, recharge the card, select an entry station, complete a journey at an exit station and review journey history. Fare is calculated automatically from the distance between the selected stations.

2.1 Project Objectives

- Provide a simple digital alternative to repeated paper-ticket purchases.
- Demonstrate Java classes, inheritance, abstraction, encapsulation and polymorphism.
- Use JDBC to connect the Java application with a relational MySQL database.
- Automate fare calculation and balance deduction during journey completion.
- Maintain persistent records for passengers, smart cards, journeys and transactions.
- Demonstrate validation and exception handling for invalid user operations.

2.2 Scope

The prototype scope includes passenger registration and login, smart-card issuance, card recharge, station listing, metro entry/exit, distance-based fare calculation and journey history. It is intentionally a terminal application and does not attempt to emulate physical NFC readers, payment gateways, live train tracking or a production-grade authentication service.

3. Problem Statement

Traditional metro ticketing can require passengers to repeatedly purchase tickets, while manual recording of journeys and fares can increase processing time and the possibility of data-entry errors. Daily passengers benefit from a reusable payment mechanism in which a stored balance can be used for multiple journeys.

The project addresses this problem by modelling a smart-card workflow. A card is associated with a passenger; an entry creates an open journey; an exit triggers distance calculation, fare determination, balance validation, fare deduction and transaction logging. The result is a consistent digital record of the trip.

3.1 Proposed Solution

1. Register or authenticate a passenger.
2. Issue or retrieve an active smart card.
3. Recharge the card when additional balance is required.
4. Record the passenger's entry station.
5. Record the exit station and calculate travel distance.
6. Calculate fare using predefined distance slabs.
7. Deduct the fare only when the card has sufficient balance.
8. Store the completed journey and fare transaction in MySQL.

4. Functional Requirements

ID	Requirement	Description
FR-01	Passenger Registration	Accept name, phone, optional email and password; validate required fields.
FR-02	Passenger Login	Authenticate using phone number and password.
FR-03	Demo Login	Provide a preloaded demo passenger for evaluation.
FR-04	Smart Card Issuance	Create an active card with a generated card number and initial balance.
FR-05	View Smart Card	Display card ID, card number, balance and status.
FR-06	Recharge	Increase card balance and record a RECHARGE transaction.
FR-07	Station Listing	Display stations with line name and distance from origin.
FR-08	Start Journey	Validate card/station and create an OPEN journey.
FR-09	End Journey	Validate exit station, calculate distance/fare, deduct balance and complete journey.
FR-10	Journey History	Display previous journeys with entry, exit, fare and status.
FR-11	Fare Information	Display the distance-to-fare slabs used by the application.
FR-12	Validation/Error Handling	Reject invalid numbers, missing fields, duplicate phone numbers, invalid cards and insufficient balance.

5. Non-functional Requirements

Attribute	Requirement
Usability	Menu-driven CLI with clear prompts and success/error messages.
Reliability	Uses validation, SQL constraints and transactions for critical journey/fare updates.
Maintainability	Business logic is separated into MetroService, fare logic and domain classes.
Performance	The prototype uses direct JDBC queries and simple in-memory lists; suitable for academic/demo scale.
Portability	Designed for Java 17+ and can be launched from command line using run.bat or run.sh.
Data Integrity	Primary keys, foreign keys, unique constraints and transactional updates protect relationships.
Security	Prototype-level only; passwords are stored plainly and production deployment would require hashing.
Scalability	Architecture separates CLI, service logic and database access so components can be replaced or extended.

6. System Architecture

The application follows a lightweight layered architecture. Main.java acts as the presentation layer and collects terminal input. MetroService contains the main business rules and coordinates database operations. Domain classes represent passengers, cards, stations and users. FareCalculator isolates the fare policy, while DatabaseConnection centralizes JDBC connectivity. MySQL stores persistent records.

6.1 Architecture Flow

- Presentation Layer → Main.java: menus, prompts, input collection and user-facing output.
- Business Layer → MetroService.java: registration, login, card, recharge and journey operations.
- Domain Layer → User, Passenger, Admin, SmartCard and Station: object representation.
- Utility/Rule Layer → FareCalculator.java and ValidationException.java.
- Data Access Layer → DatabaseConnection.java and JDBC PreparedStatements.
- Database Layer → MySQL database named metro_system.

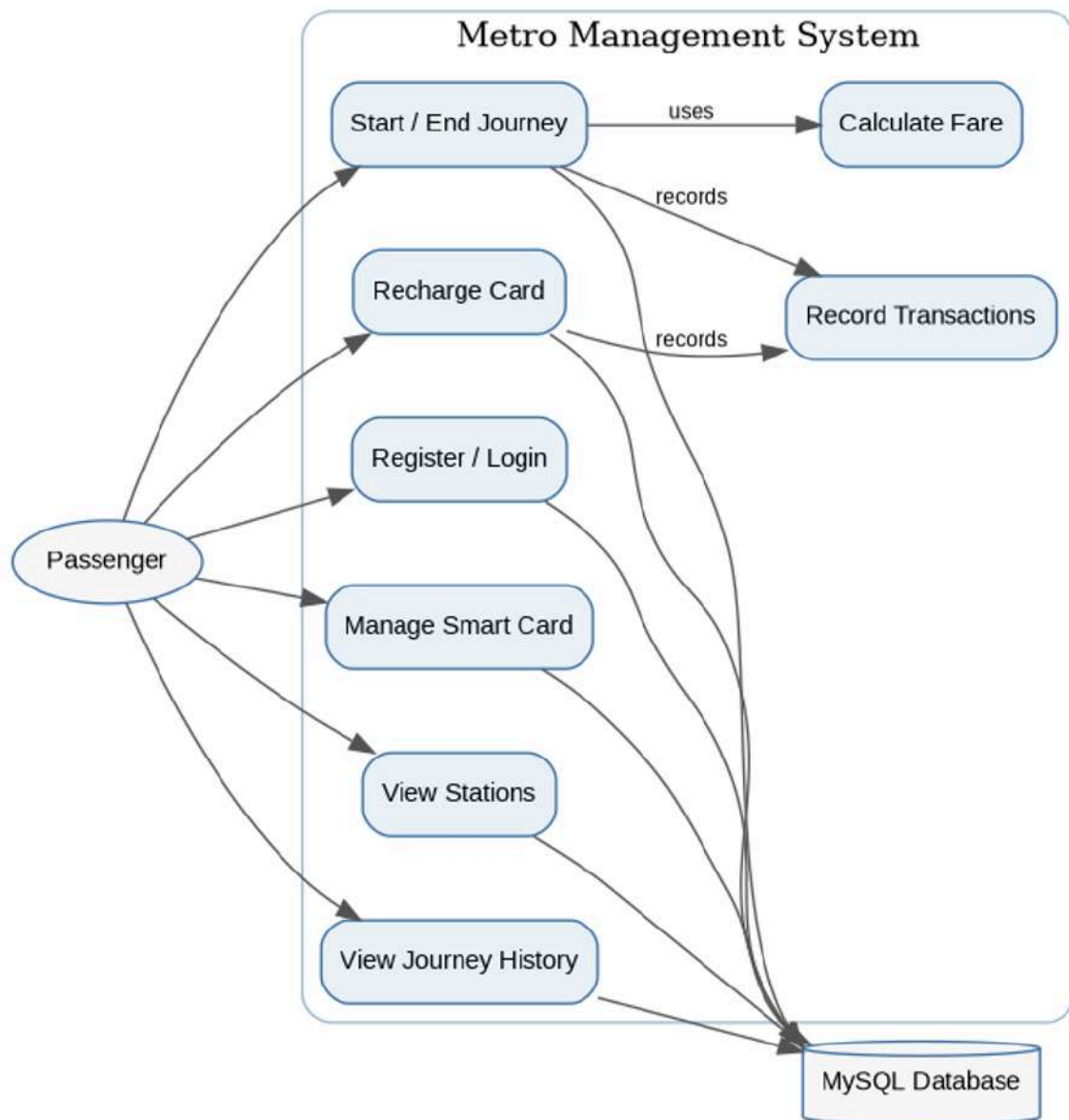
6.2 Technology Stack

Technology	Role
Java 17+	Application language and object-oriented implementation.
Core Java	CLI, collections, exception handling and object model.
JDBC	Database connectivity and parameterized SQL execution.
MySQL 8+	Persistent relational data storage.
MySQL Connector/J	JDBC driver required to connect Java to MySQL.
Command Line	User interface and project evaluation environment.

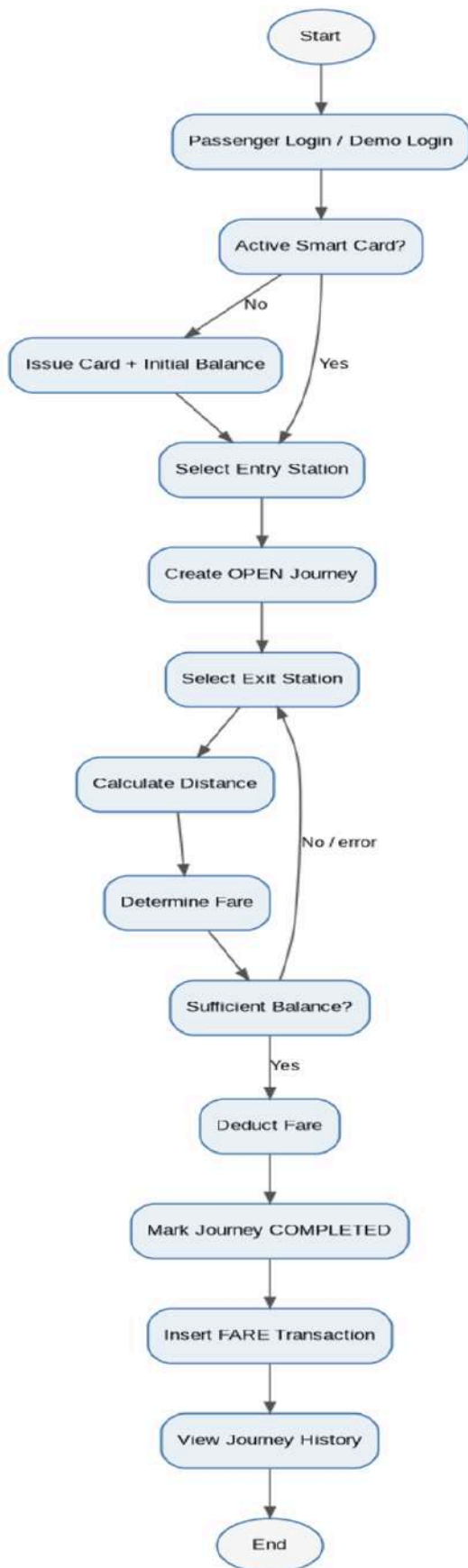
7. Design Diagrams

The diagrams below document the system from different perspectives: user interactions, process flow, message sequence, software structure and relational data design.

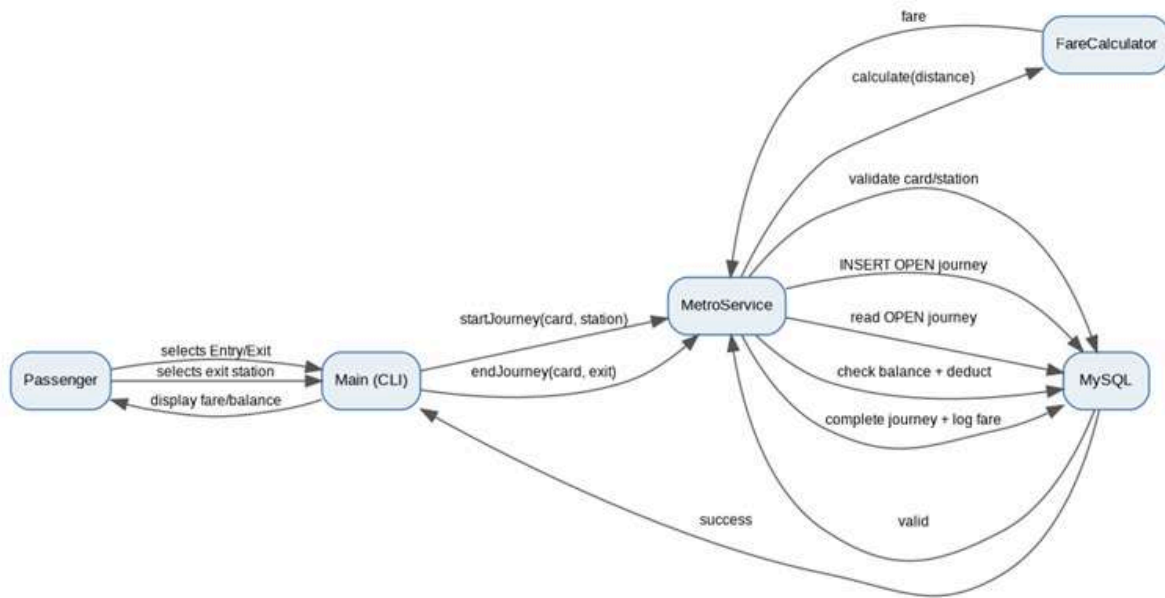
7.1 Use Case Diagram



7.2 Workflow Diagram

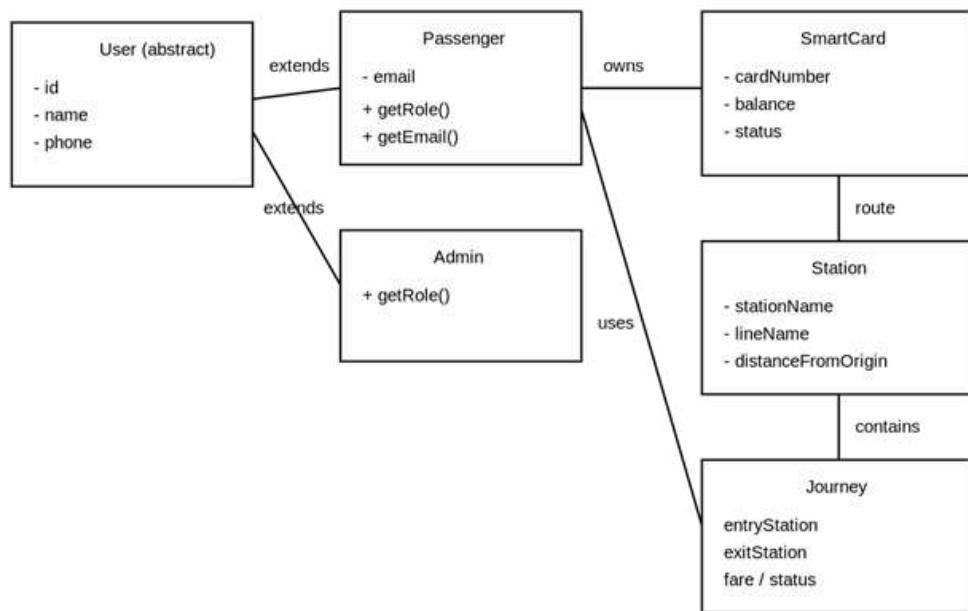


7.3 Sequence Diagram



7.4 Class/Component Diagram

Metro Management System - UML/Class Diagram



7.5 ER Diagram



8. Design Decisions & Rationale

Decision	Rationale
CLI instead of GUI	The project is intentionally executable from a terminal, reducing setup complexity and making core Java logic easy to evaluate.
Service layer	MetroService centralizes business rules so Main.java remains focused on interaction and display.
PreparedStatement	Parameterized SQL reduces SQL-injection risk and keeps database calls structured.
Database transactions	Recharge and journey completion use transaction control so related updates can commit together or roll back on failure.
Distance-based fare	Station distance values provide a simple deterministic basis for the academic fare model.
Generated card number	UUID-derived identifiers avoid dependence on manually entered card numbers.
Custom ValidationException	Domain-specific validation errors can be shown as readable CLI messages.
Relational schema	Separate tables for passengers, cards, stations, journeys and transactions preserve data relationships and history.

9. Implementation Details

9.1 Main Classes

Class/File	Responsibility
Main.java	CLI entry point, menus, input collection and user-facing output.
MetroService.java	Registration, login, smart card, recharge, station and journey business logic.
DatabaseConnection.java	Creates JDBC connections to MySQL.
User.java	Abstract base class containing common user attributes and getRole().
Passenger.java	Passenger specialization with optional email.
Admin.java	Admin specialization of User.
SmartCard.java	Card number, balance, status and card ID.
Station.java	Station identity, line and distance from origin.
FareCalculator.java	Converts journey distance into fare slabs.
ValidationException.java	Custom checked exception for invalid operations.

9.2 Fare Calculation Logic

Distance	Fare
0–5 km	₹10
>5–10 km	₹20
>10–20 km	₹30
>20 km	₹40

During endJourney(), the application reads the entry and exit station distances, takes the absolute difference, passes that value to FareCalculator.calculate(), checks the card balance and then performs the balance deduction, journey completion and transaction insertion inside a database transaction.

9.3 Database Schema

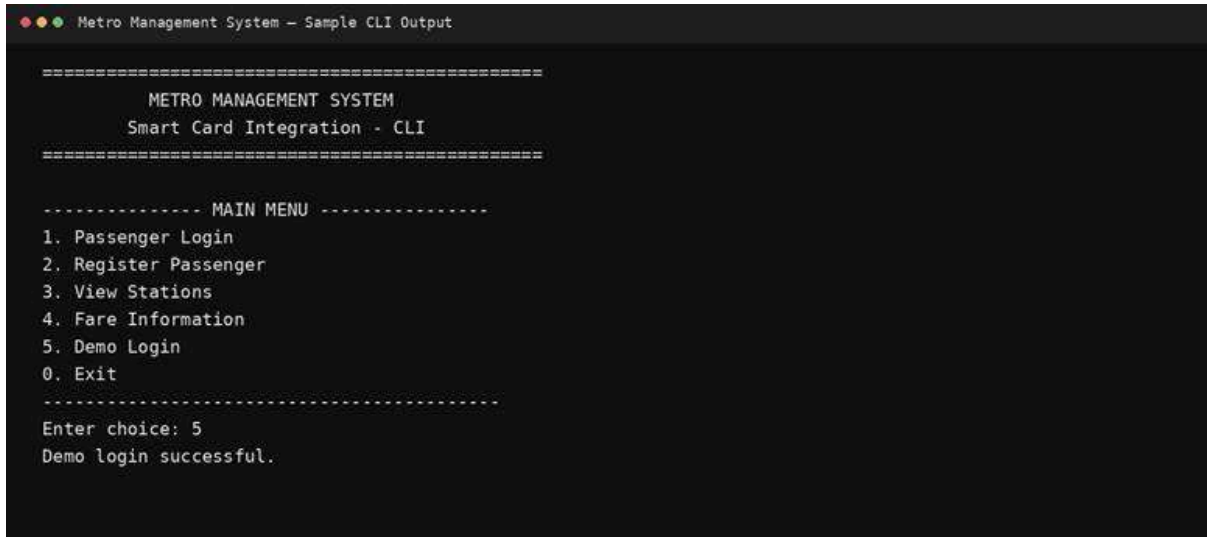
Table	Purpose	Important Relationships
admins	Stores administrator credentials for future administrative features.	Standalone in current prototype.
passengers	Stores passenger account data.	One passenger can have smart cards.
stations	Stores station name, line and distance.	Referenced by journey entry/exit.
smart_cards	Stores card number, balance and status.	Belongs to passenger; referenced by journeys/transactions.
transactions	Stores recharge and fare records.	Belongs to a smart card.
journeys	Stores entry/exit stations, timestamps, fare and status.	References card and stations.

9.4 Project Structure

```
MetroManagementSystem_CLI/
├── src/metro/
│   ├── Main.java
│   ├── MetroService.java
│   ├── DatabaseConnection.java
│   ├── User.java
│   ├── Passenger.java
│   ├── Admin.java
│   ├── SmartCard.java
│   ├── Station.java
│   ├── FareCalculator.java
│   └── ValidationException.java
├── sql/metro_system.sql
├── docs/
│   ├── PROJECT_REPORT.md
│   ├── Project_Report.docx
│   ├── Metro_Project_PPT.pptx
│   └── UML_Class_Diagram.svg
├── run.bat
└── run.sh
```

10. Screenshots / Results

The following five terminal-output figures document the principal user-visible results of the project. They are representative CLI outputs based directly on the application's implemented menus, messages and the demo records included in metro_system.sql.

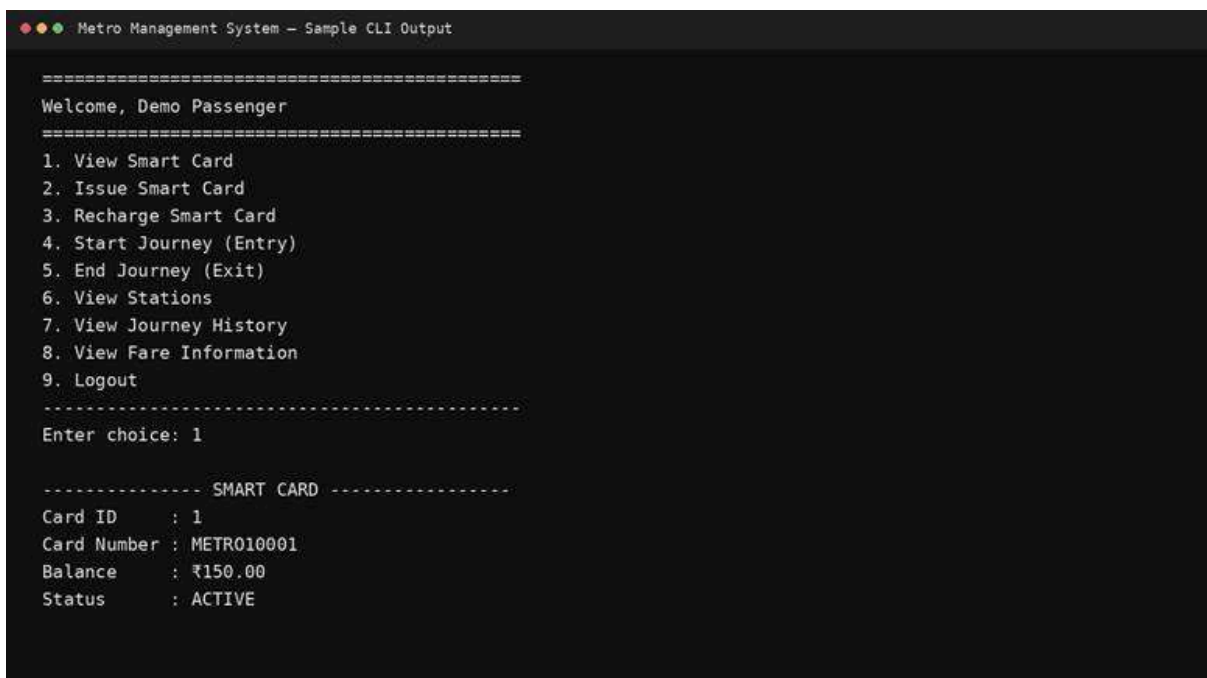


```
Metro Management System - Sample CLI Output

=====
METRO MANAGEMENT SYSTEM
Smart Card Integration - CLI
=====

----- MAIN MENU -----
1. Passenger Login
2. Register Passenger
3. View Stations
4. Fare Information
5. Demo Login
0. Exit
-----
Enter choice: 5
Demo login successful.
```

Figure 10.1 — Main menu and demo login.



```
Metro Management System - Sample CLI Output

=====
Welcome, Demo Passenger
=====
1. View Smart Card
2. Issue Smart Card
3. Recharge Smart Card
4. Start Journey (Entry)
5. End Journey (Exit)
6. View Stations
7. View Journey History
8. View Fare Information
9. Logout
-----
Enter choice: 1

----- SMART CARD -----
Card ID      : 1
Card Number  : METRO10001
Balance      : ₹150.00
Status       : ACTIVE
```

Figure 10.2 — Passenger menu and active smart-card details.

```
Metro Management System - Sample CLI Output

----- METRO STATIONS -----
ID      Station      Line      Distance
-----
1       Bhopal Central  Blue Line  0.00 km
2       MP Nagar        Blue Line  4.00 km
3       Habibganj         Blue Line  7.00 km
4       Misrod            Blue Line  12.00 km
5       Mandideep         Blue Line  20.00 km

----- FARE INFORMATION -----
Distance      Fare
0 - 5 km      ₹10
Above 5 - 10 km  ₹20
Above 10 - 20 km ₹30
Above 20 km    ₹40
```

Figure 10.3 — Station listing and fare information.

```
Metro Management System - Sample CLI Output

Enter choice: 4

----- METRO STATIONS -----
ID      Station      Line      Distance
-----
1       Bhopal Central  Blue Line  0.00 km
2       MP Nagar        Blue Line  4.00 km
3       Habibganj         Blue Line  7.00 km
4       Misrod            Blue Line  12.00 km
5       Mandideep         Blue Line  20.00 km
Enter entry station ID: 2
Entry recorded successfully.
Remember to end the journey at your destination station.

Enter choice: 5
Enter exit station ID: 4
Journey completed successfully.
Fare deducted : ₹20.00
New balance   : ₹130.00
```

Figure 10.4 — Journey entry/exit and automatic fare deduction.

```
Metro Management System - Sample CLI Output

----- JOURNEY HISTORY -----
Entry Time      Entry      Exit      Fare      Status
-----
2026-09-18 21:45:12  MP Nagar  Misrod    ₹20.00    COMPLETED

Recharge successful. New balance: ₹150.00

Thank you for using Metro Management System!
```

Figure 10.5 — Journey history and completion output.

11. Testing Approach

Testing is organized around functional scenarios and validation cases. The aim is to verify both successful operations and the error-handling paths implemented in MetroService and Main.

ID	Test	Input/Scenario	Expected Result
TC-01	Demo login	Use preloaded demo credentials	Passenger menu opens
TC-02	Registration validation	Blank name/phone/password	Validation message is displayed
TC-03	Phone validation	Phone with non-digit or invalid length	Registration is rejected
TC-04	Duplicate phone	Register an existing phone number	Database constraint is converted to readable validation error
TC-05	Card display	View active demo card	Card number, balance and ACTIVE status shown
TC-06	Recharge	Recharge with positive amount	Balance increases and RECHARGE transaction is stored
TC-07	Recharge validation	Recharge with zero/negative value	Operation rejected
TC-08	Start journey	Select valid entry station	OPEN journey created
TC-09	Duplicate open journey	Start another journey before exit	Operation rejected
TC-10	End journey	Select valid exit station with sufficient balance	Fare calculated, balance deducted, journey completed
TC-11	Insufficient balance	Attempt journey with balance below fare	Operation rejected and transaction is rolled back
TC-12	History	View journey history	Completed/open journeys are displayed
TC-13	CLI help	Run Main with --help	Usage instructions displayed

11.1 Test Strategy

- Positive testing for normal passenger workflows.
- Boundary testing for fare slabs at 5, 10 and 20 km.
- Validation testing for missing fields and invalid numeric input.
- Database integrity testing through unique and foreign-key constraints.
- Transaction testing for recharge and end-journey operations.
- Regression checks after changes to fare, card and journey logic.

12. Challenges Faced

- Coordinating console input with database operations while keeping user messages clear.
- Maintaining consistent smart-card state across registration, issuance, recharge and journey operations.
- Ensuring that an open journey cannot be duplicated for the same card.
- Keeping fare deduction, journey completion and transaction logging consistent through database transactions.
- Handling invalid user input without terminating the CLI application.
- Balancing academic simplicity with production concerns such as password security and physical smart-card integration.

12.1 Resolution Approach

The implementation separates business operations into MetroService, uses ValidationException for expected validation failures, uses PreparedStatement for SQL calls and uses commit/rollback around critical multi-step operations. This keeps the main menu readable and makes the core workflow easier to test.

13. Learnings & Key Takeaways

- Object-oriented design becomes clearer when domain entities and responsibilities are separated into classes.
- Inheritance and abstraction can reduce duplication for related user types.
- JDBC provides a direct way to connect Java business logic with persistent relational data.
- Prepared Statements and database constraints are important even in small academic applications.
- Transactions are necessary when one user action changes several related records.
- Input validation is part of application design, not an optional add-on.
- A simple CLI can still demonstrate complete application flow when the requirements are well structured.
- Clear diagrams help communicate the same system at user, process, software and data levels.

14. Future Enhancements

Enhancement	Description
NFC/Smart-card hardware	Connect physical readers so entry and exit can be triggered by a real card.
QR ticketing	Add QR-based temporary tickets for users without reusable cards.
Online payment gateway	Allow secure digital recharge through UPI/cards/net banking.
Mobile application	Expose registration, balance, recharge and journey history through Android/iOS.
Monthly passes	Add pass products and configurable eligibility/validity rules.
Real-time train tracking	Integrate live train positions, platform information and arrival estimates.
Admin dashboard	Add station, passenger, card, fare and transaction administration.
Secure authentication	Replace plain-text passwords with salted hashing and secure session management.
Cloud deployment	Move the database and service layer to managed cloud infrastructure.
Analytics	Generate usage, revenue, peak-hour and station-demand reports.

15. References

1. Project source code — Main.java, MetroService.java, DatabaseConnection.java, FareCalculator.java and domain classes included in the submitted project archive.
2. Project database script — sql/metro_system.sql, containing schema definitions, relationships and demo data.
3. Project README — README.md, containing project overview, technology stack, setup and command-line execution guidance.
4. UML class diagram — docs/UML_Class_Diagram.svg included in the submitted project archive.
5. Oracle Java Documentation — Java language and standard library concepts used by the project.
6. MySQL Documentation — relational database, SQL, constraints and transaction concepts relevant to the implementation.
7. MySQL Connector/J Documentation — JDBC driver and Java-to-MySQL connectivity reference.

